

Python 3 Programming Primer: Basics to Intermediate

A compact, practice-oriented guide covering variables, control flow, loops, functions, built-in data structures, files, modules, classes, typing, errors, iterators/generators, and common libraries—with examples and exercises from beginner to intermediate level.

0) Setup & Run

Install: <https://www.python.org/downloads/> (ensure Python ≥ 3.10)

Create a virtual environment

```
python3 -m venv .venv
source .venv/bin/activate    # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
```

Run a script

```
python hello.py
```

REPL: run python then type code interactively. Quit with `exit()` or Ctrl-D.

1) Hello, World

```
print("Hello, World!")
```

Exercise 1.1: Print your name and the current year on separate lines using one print call.

2) Variables, Types, and Basic I/O

Dynamic typing; names bind to objects.

```
age = 30          # int
price = 9.99      # float
name = 'Ana'      # str (use single or double quotes)
is_student = False # bool
```

Input/Output

```
x = int(input('Enter an integer: '))
y = float(input('Enter a float: '))
print(f'x={x} y={y:.2f}')
```

Type conversion: `int()`, `float()`, `str()`, `bool()`.

Exercise 2.1: Compute the area and circumference of a circle from a user-given radius. Format to 2 decimals.

3) Operators & Expressions (quick tour)

- Arithmetic: + - * / // % **
- Comparisons: == != < <= > >=
- Logical: and or not
- Assignment: = += -= *= /= //= %= **=
- Membership: in, not in
- Identity: is, is not

Exercise 3.1: Given integer n, print whether it is even or odd using % and a conditional expression.

4) Control Flow

if / elif / else

```
score = 87
if score >= 90:
    print('A')
elif score >= 80:
    print('B')
else:
    print('C or below')
```

for & while loops

```
for i in range(5):
    print(i, end=' ')
```

```
j = 5
while j > 0:
    print(j, end=' ')
    j -= 1
```

break / continue / else on loops

```
for x in range(2, 10):
    if x % 2 == 0:
        continue
    print('odd:', x)
else:
    print('no break encountered')
```

Exercise 4.1: Print the first n Fibonacci numbers.

Exercise 4.2: Number-guessing game with while and break.

5) Collections: list, tuple, dict, set

list

```
nums = [1, 2, 3]
nums.append(4)
nums.extend([5, 6])
nums[1] = 99
for v in nums:
    print(v)
```

tuple (immutable, useful for fixed records)

```
pt = (10, 20)
x, y = pt
```

dict (key→value map)

```
ages = {'Ana': 30, 'Ben': 28}
ages['Cara'] = 33
for name, age in ages.items():
    print(name, age)
```

set (unique elements)

```
s = {1, 2, 2, 3}
print(s)          # {1, 2, 3}
print({1,2} | {2,3}, {1,2} & {2,3})
```

Comprehensions

```
squares = [x*x for x in range(10) if x % 2 == 0]
lookup = {c: ord(c) for c in 'abc'}
```

Exercise 5.1: Read a line and count vowels (a,e,i,o,u) case-insensitive.

Exercise 5.2: Given a list of (name, score) tuples, return top-3 by score.

6) Strings

```
s = " Hello, Ana! "
print(s.strip().lower())
print(s.replace('Ana', 'Ben'))
print('ana' in s.lower())
```

Formatting

```
user = 'Ana'; gpa = 6.5
print(f'{user} has GPA {gpa:.2f}')
```

Exercise 6.1: Implement `reverse_string(s: str) -> str`.

7) Functions & Modules

```
def add(a: int, b: int) -> int:
    return a + b

def greet(who: str, /, *, excited: bool = False) -> None:
    msg = f"Hi, {who}!" + (' 🐼' if excited else '')
    print(msg)

if __name__ == '__main__':
    print(add(2, 3))
    greet('world', excited=True)
```

Docstrings

```
def mean(xs: list[float]) -> float:
    """Return the arithmetic mean of xs; xs must be non-empty."""
    return sum(xs) / len(xs)
```

Importing

```
# file: mathx.py
PI = 3.14159

def area_circle(r: float) -> float:
    return PI * r * r

# file: main.py
import mathx
print(mathx.area_circle(2))
```

Exercise 7.1: Write `is_prime(n)` and a `primes_up_to(n)` generator (see §11).

8) Exceptions & Error Handling

```
def safe_div(a: float, b: float) -> float:
    try:
        return a / b
    except ZeroDivisionError as e:
        print('cannot divide by zero:', e)
```

```

        return float('inf')
    finally:
        pass # cleanup here if needed

```

Custom exception

```

class ConfigError(Exception):
    pass

```

Exercise 8.1: Implement a function that parses "key=value" lines and raises on duplicates.

9) Files & Paths

Prefer pathlib and context managers.

```

from pathlib import Path
p = Path('data.txt')
p.write_text('42 answer\n', encoding='utf-8')
text = p.read_text(encoding='utf-8')
print(text)

```

```

# Line by line
with p.open() as f:
    for line in f:
        print(line.rstrip())

```

CSV & JSON

```

import csv, json

rows = [{ 'id': 1, 'name': 'Ana', 'value': 6.5 }, { 'id': 2, 'name': 'Ben',
'value': 5.9 } ]
Path('data.json').write_text(json.dumps(rows, indent=2), encoding='utf-8')

with open('data.csv', 'w', newline='') as f:
    w = csv.DictWriter(f, fieldnames=['id', 'name', 'value'])
    w.writeheader(); w.writerows(rows)

```

Exercise 9.1: Copy a text file to another, counting lines and bytes.

10) Classes, Dataclasses & OOP

```

class Student:
    def __init__(self, sid: int, name: str, gpa: float):
        self.sid = sid; self.name = name; self.gpa = gpa
    def __repr__(self) -> str:

```

```

        return f"Student({self.sid!r}, {self.name!r}, {self.gpa!r})"
    def pretty(self) -> str:
        return f"{self.sid} {self.name} {self.gpa:.2f}"

```

dataclasses

```
from dataclasses import dataclass
```

```

@dataclass(order=True)
class Row:
    value: float
    name: str

```

Exercise 10.1: Model a Point with methods `distance(self, other)` and `translate(dx, dy)`.

11) Iterables, Iterators & Generators

```

def countdown(n: int):
    while n > 0:
        yield n
        n -= 1

```

```

for x in countdown(3):
    print(x)

```

Generator expressions

```
total = sum(x*x for x in range(10))
```

itertools

```

from itertools import islice, pairwise
print(list(islice(range(100), 5)))
print(list(pairwise([1,2,3,4]))) # Python 3.10+

```

Exercise 11.1: Implement an infinite generator of Fibonacci numbers and print the first 30.

12) Worked Examples

19.1 Temperature Converter

```
c = float(input('Celsius: '))
f = c * 9/5 + 32
print(f'Fahrenheit: {f:.2f}')
```

19.2 Histogram of Word Lengths

```
import sys
import string

hist = {k:0 for k in range(1, 21)}
count = 0
for ch in sys.stdin.read():
    if ch.isalpha():
        count += 1
    elif count:
        hist[min(count, 20)] += 1
        count = 0
for k in range(1, 21):
    print(f'{k:2d}: {hist[k]}')
```

19.3 Tiny CSV Reader into Objects

```
from dataclasses import dataclass
from pathlib import Path
import csv

@dataclass
class Row:
    id: int
    name: str
    value: float

rows: list[Row] = []
with open('data.csv', newline='') as f:
    for r in csv.DictReader(f):
        rows.append(Row(int(r['id']), r['name'], float(r['value'])))

for r in sorted(rows, key=lambda r: (-r.value, r.name)):
    print(r)
```

13) Progressive Exercise Set (with hints)

Beginner 1. Read an int `n` and print the sum from 1..`n`. (*Loop, int arithmetic*) 2. Read three integers and output the maximum. (*if/elif*) 3. Read a line, print it in uppercase. (*str.upper*)

Intermediate A 4. Implement binary search on a sorted list. (*indices*) 5. Implement `string_to_int(s)` handling optional `+/ -` and digits. (*iteration, exceptions*) 6. Build a `Complex` class with `add`, `mul`, and `abs` (magnitude). (*OOP*)

Intermediate B 7. Implement a dynamic vector of floats with methods: `push`, `pop`, `get`, `set`, `resize`. (*list wrapper*) 8. Implement a singly linked list of integers with `push_front`, `insert_after`, `remove`, `find`, and `__iter__`. (*classes*) 9. Parse a text file of "name score" pairs into objects and print the top-k. (*file I/O, sorting*)

Intermediate C 10. Implement a `Matrix` class supporting `shape`, `get`, `set`, and `mul` (naïve). (*nested lists*) 11. Build a CLI: `calc op a b` using `argparse`. (*\$16*) 12. Implement a mini JSON-like parser for `{key:value}` pairs with no nesting. (*strings, state machine*)

14) Reading List & Next Steps

- *Automate the Boring Stuff with Python* (A. Sweigart)
 - *Fluent Python* (L. Ramalho, 2nd ed.)
 - *Effective Python* (B. Slatkin, 2nd ed.)
-